노트북에서 서비스까지: MLOps와 클라우드 기반 AI 운영 흐름

MLOps 기초 및 커리어 이야기

발표자 소개

김보영

Cloud Engineer

미국계 퍼블릭 클라우드 밴더 근무중

경력 하이라이트

- 컴퓨터공학 학사, 7년차 엔지니어
- 빅테크, 스타트업을 포함한 글로벌 기업 3곳에서 근무
- 클라우드 환경에서 운영 및 고객 지원 경험
- DevOps Engineer 로서의 개발 및 운영 경험

전문 분야

- DevOps
- Kubernetes
- 고객 커뮤니케이션
- 가상화
- 성능 최적화
- IaC (Infrastructure as a Code)
- Cloud (AWS/GCP/Azure)
- CI/CD
- BigData Platform 운영
- Observability & Monitoring

오늘의 여정 (5단계 로드맵)

01

문제 정의

데이터팀이 만든 모델을 실제 서비스 앱에 어떻게 연결할지 고민하는 단계입니다.

02

실무 배경

이론보다는 실제 현업 프로젝트에서 왜 MLOps가 필요한지 배경을 이해합니다.

03

라이프사이클 플로우

데이터 수집부터 모델 배포, 모니터링, 재학습까지 이어지는 전체 순환 구조(Loop)를 파악합니다.

End-to-End

Feedback Loop

04

클라우드 실전

클라우드 및 쿠버네티스 환경에서 기술적 구현 방법과 아키텍처를 살펴봅니다.

05

커리어 성장

오늘 배운 내용이 여러분의 커리어 성장에 어떤 강력한 무기가 될지 확인합니다.

오늘 가져갈 3가지 목표

전체 흐름 이해

노트북 → 클라우드 → 운영 → 재학습
서비스 라이프사이클을 한 눈에 이해하기

역할 이해

데이터 사이언티스트, 백엔드/ML, 클라우드, 프론트엔드가
어디서 어떻게 연결되는지 이해하기

커리어 방향

개인의 관심사에 따라
어떤 스킬을 보완할지 로드맵 잡기

Goal of this Journey

전체 흐름을 시각적으로 이해하고, 여러분의 역할을 명확히 하는 것입니다.

시작하기

프로덕션 배포 경험?

모델을 실제 서비스에 배포해 본 적이 있나요?

Docker/Kubernetes?

들어본 적이 있나요,
아니면 익숙하게 다루시나요?

잘 몰라도 괜찮나요?

네, 괜찮습니다!
오늘은 “무슨 역할을 하는가”에 집중합니다.

사례 연구: 추천 시스템, 앱에 연결하기

데이터팀이 만든 모델을 e-커머스 앱에 붙이는 3단계 타임라인

컨텍스트

추천 모델
데이터팀이 사용자별 상품 추천 모델을 개발

목표
e-커머스 웹/모바일 앱에서 안정적으로 예측 호출

연결 포인트
React/React Native → API Gateway → ML 서비스 (e.g. FastAPI)

핵심: 프론트는 **/predict**를 호출하고, 뒤에서는 모델·인프라가 협업합니다.

3단계 타임라인

노트북 → API/배포 → 운영/재학습

Day 1–30
노트북에서 모델 개발 전처리 및 실험 정리

Day 31–60
FastAPI로 /predict 래핑 Docker로 컨테이너화 및 클라우드 배포

Day 61+
성능·드리프트 모니터링 재학습·버전 관리·롤아웃

주의 포인트: 노트북과 서비스의 **전처리 일치**, **응답 시간** 예산, **모델 성능 저하** 모니터링

실제 발생한 문제점 3가지

개발 · 배포 · 운영 단계에서의 대표 이슈와 원인

개발 단계

증상

로컬(노트북)에서는 잘 되는데 클라우드 서빙 결과가 다름

원인

입력 전처리 로직 불일치(노트북 vs 서버)

라이브러리/모델 버전 차이, 시드(Seed) 고정 누락

대응: 전처리 코드를 패키지화/공유, 환경(versions) 고정

배포 단계

증상

API 목표 1초인데, 사용자 체감 3초 이상

원인

모델 로드 지연(2초), 네트워크 지연(0.5초), 콜드스타트

싱글 인스턴스 병목, 비효율적 직렬 처리

대응: 사전 로드, 캐싱/배치, 오토스케일링, 경량화(ONNX 등)

운영 단계

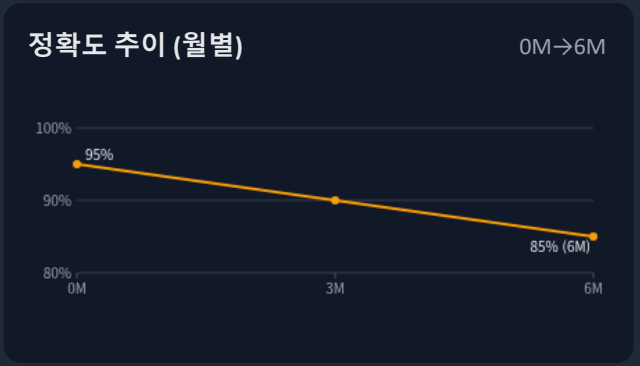
증상

정확도 95% → 6개월 후 85%로 하락

원인

데이터 드리프트(입력 분포 변화), 계절성/트렌드 변화

라벨 지연, 피처 스키마 변경 미감지



대응: 모니터링/알림, 주기적 재학습, 버전 관리·롤백

팀별로 수행하는 역할

DS · 백엔드 · MLOps · 프론트엔드 · SRE의 담당 업무와 연결 지점

데이터 사이언티스트
노트북·모델 실험
- 데이터 탐색 & 분석
- 피처 엔지니어링
- 모델 학습·평가
- 실험 추적(MLflow)

백엔드 엔지니어
API 개발·서비스 연동
- FastAPI/API 개발
- 모델 서빙 연동
- 데이터 전처리 파이프라인
- 캐싱/성능 최적화

MLOps / 플랫폼
인프라·자동화
- Docker/Container
- K8s 오케스트레이션
- CI/CD 파이프라인
- 모델 레지스트리

프론트엔드 / 앱
UX·클라이언트
- AI 기반 UX 구현
- 비동기 API 처리
- 결과 시각화
- 피드백 수집

SRE / DevOps
신뢰성·운영
- 모니터링 (SLA/SLO)
- 오토스케일링
- 장애 대응
- 보안 관리

→ **요청 흐름 :** App → API Gateway → **Backend/ML Service** → Model Store | **루프 :** 모니터링 → 재학습 → 검증 → **배포**

MLOps란 무엇인가?

한 문장으로 이해하는 MLOps의 정의

> **MLOps는 머신러닝 모델을 개발 → 배포 → 운영 → 재학습까지 안정적이고 반복 가능하게 만들기 위한 방법론과 도구 세트입니다.**

핵심은 **소프트웨어 엔지니어링(DevOps)의 성공적인 관행**을 **ML 시스템의 특수성**에 맞춰 적용하는 것

- 모델·데이터 버전 관리
- CI/CD/CT 자동화
- 지속적 모니터링
- 재현 가능한 환경

DevOps vs MLOps vs ML

DevOps
소프트웨어 배포 자동화

초점

CI/CD 파이프라인,
릴리즈 자동화

인프라 관리
(IaC),
모니터링/알림

대표 도구

GitHub Actions | Jenkins | Terraform | Prometheus

CORE

MLOps
DevOps + 데이터·모델 관리

초점

모델/데이터 버전 관리,
실험 추적

서빙/배포 파이프라인,
성능·드리프트 모니터링

피드백 기반 재학습·롤아웃

대표 도구

MLflow | Kubeflow | SageMaker | KServe

ML
알고리즘·학습·평가

초점

데이터 전처리,
피처 엔지니어링

모델 아키텍처 설계,
학습 및 하이퍼파라미터 튜닝

평가 지표 최적화 (AUC 등)

대표 도구

scikit-learn | PyTorch | TensorFlow | Pandas

요약: **MLOps**는 DevOps의 운영 관행에 ML 데이터·모델의 특수성을 결합하여, 지속 가능하고 안정적인 AI 서비스를 만듭니다.

MLOps가 필요한 5가지 이유

모델 성능 저하 · 환경 차이 · 버전 관리 · 모니터링 · 팀 협업

모델 성능 저하 (Data Drift)

시간이 지나면 현실 세계의 데이터 분포가 바뀌어 정확도가 떨어집니다. 코드는 그대로여도 성능이 하락하는 유일한 소프트웨어입니다.

환경 차이 (Training vs Serving)

"내 노트북에선 되는데?" 문제를 해결해야 합니다. 로컬의 유연한 실험 환경과 클라우드의 엄격한 배포 환경 간의 격차를 줄입니다.

3차원 버전 관리

일반 SW는 코드만 관리하지만, ML은 [코드 + 데이터 + 모델 가중치] 3박자가 맞아야 과거의 결과를 완벽히 재현할 수 있습니다.

침묵하는 실패 감지

서버가 200 OK를 반환해도 엉뚱한 예측을 할 수 있습니다. 지연시간뿐만 아니라 모델의 정확도 변화를 실시간으로 감시해야 합니다.

사일로 파괴 (Silo Breaking)

실험 중심의 데이터 사이언티스트와 안정성 중심의 엔지니어가 파이프라인이라는 공통 언어로 협업하게 만듭니다.

MLOps 라이프사이클 (루프)

데이터&실험 → 학습&패키징 → 배포&서빙 → 모니터링&피드백 (지속적 순환)

데이터 & 실험

데이터 수집, 노트북 실험, 피처 엔지니어링

학습 & 패키징

모델 학습·평가, 아티팩트 저장(예: pkl/pt/onnx)

배포 & 서빙

컨테이너화, API 오픈, K8s/서버리스로 확장

모니터링 & 피드백

지연·에러·정확도·드리프트 감시, 재학습 트리거

핵심: 한 번의 출시가 아니라 모니터링 결과에 따라 학습·배포가 반복되는 "**살아있는 시스템**"

루프가 반복되는 이유

모니터링 → 재학습 트리거 → 검증/배포 → 정확도 회복 (지속적 순환)

1) 모니터링 이상 감지

정확도(Accuracy) 급락이나 데이터 분포 변화(Drift)를 실시간으로 포착합니다.

2) 재학습 파이프라인 트리거

최신 데이터를 포함하여 모델을 다시 학습시키고, 새로운 아티팩트를 생성합니다.

3) 검증(Validation)과 승인

새 모델이 기존 모델보다 성능이 우수한지 자동 평가 후 배포 승인을 내립니다.

4) 배포 및 성능 회복

서비스에 적용하여 떨어진 정확도를 다시 끌어올립니다. (Loop 완성)

모델 성능 추이 시뮬레이션

Live Metric

T1 시점	T2~T3 구간	T4 시점
드리프트 발생	재학습/검증	성능 회복

모델의 변신 과정 (5단계)

노트북 코드 → 모델 아티팩트 → 서빙 API → 컨테이너 이미지 → 배포된 서비스

1) 노트북 코드
Data Scientist

개발자 로컬 환경에서 Jupyter로 실험 및 모델 개발

2) 모델 아티팩트
Data Scientist

pkl/pt/onnx 파일로 저장 및 버전 관리 (Reproducibility)

3) 서빙 API
ML Engineer

FastAPI 등으로 /predict 엔드포인트 래핑

4) 컨테이너 이미지
MLOps Engineer

Docker로 코드, 모델, 의존성 라이브러리 패키징

5) 배포된 서비스
MLOps/SRE

K8s/Cloud Run/Lambda로 공개 및 운영

serving_api.py (Step 3)

```python
from fastapi import FastAPI
import pickle

app = FastAPI()
# 모델 아티팩트 로드 (Step 2 결과물)
model = pickle.load(open("model.pkl","rb"))

@app.post("/predict")
def predict(features: dict):
    x = [features.get("age"), features.get("income")]
    y = model.predict([x])[0]
    return {"prediction": y}
```

Dockerfile (Step 4)

```dockerfile
FROM python:3.9-slim
WORKDIR /app

# 의존성 설치
COPY requirements.txt .
RUN pip install -r requirements.txt
```

API 래핑 코드 상세

FastAPI로 모델을 감싸고, 프론트엔드에서 안전하게 호출하는 방법

서빙 API (FastAPI, 모델 로딩은 앱 시작 시 1회)

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel
import joblib

app = FastAPI(title="ml-service")
model = joblib.load("model.pkl")  # 앱 시작 시 1회 로드

class Features(BaseModel):
    age: int
    income: float

@app.post("/predict")
def predict(features: Features):
    x = [[features.age, features.income]]
    y = model.predict(x)[0]
    return {"prediction": float(y)}  # JSON 직렬화 안전
```

핵심 1. 모델은 **전역**에서 1회 로드 핵심 2. **Pydantic**으로 입력 검증 핵심 3. 응답은 **JSON** 규격

흐름: 앱이 /predict로 JSON 요청 → API 게이트웨이/Ingress → FastAPI에서 예측 후 JSON 응답

💡 프로덕션 환경에서는 lifespan 이벤트 사용 권장

프론트엔드 연결 예시 (fetch)

```
// React/React Native
const res = await fetch("/predict", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ age: 25, income: 50000 })
});
const data = await res.json();
// data = { prediction: 0.83 }
```

요청 스키마

```
{ "age": number,
"income": number }
```

응답 스키마

```
{ "prediction": number }
```

App — API Gateway/Ingress — FastAPI + Model

💡 프로덕션 환경에서는 **lifespan** 이벤트 사용 권장

배포 옵션 스펙트럼

복잡도(→)와 유연성/성능(↑) 축에서 4가지 옵션 비교

유연성/성능 ↑

관리형 ML 서빙

SageMaker / Vertex / AML

- A/B·배치·드리프트 등 ML 특
- 비용·클라우드 락인

Kubernetes

EKS / GKE / AKS

- 최대 유연성·표준·확장성
- 운영 복잡도·학습 곡선 큼

관리형 컨테이너

Cloud Run / Fargate

- 간편 배포, 자동 스케일
- 세밀 제어는 제한적

Serverless

Lambda / Cloud Functions

- 간단·빠른 시작, 자동 스케일
- 런타임 제한, 콜드스타트

간단한 API | 중간 규모 서비스 | 대규모·복잡 요구사항

복잡도 →

Kubernetes를 배우는 이유

클라우드 네이티브 시대의 운영 표준이자, 모든 인프라를 관통하는 핵심 원리입니다.

한 번 제대로 배우면, 어디서나 통하는 “배포·운영의 공통 언어”

AWS(EKS) · Azure(AKS) · GCP(GKE) · On-Premise 어디서든 동일한 아키텍처로 서비스를 운영할 수 있습니다.

업계 표준 · 생태계

CNCF(Cloud Native Computing Foundation)를 중심으로 한 거대한 오픈소스 생태계가 표준을 만듭니다.

Major CNCF Tools

- Prometheus
- Grafana
- Istio
- ArgoCD
- Helm

개념 전이 · 매핑

K8s의 리소스 모델을 이해하면 다른 매니지드/서버리스 서비스도 쉽게 매핑하여 사용할 수 있습니다.

Concept Mapping

Pod	→	ECS Task, Cloud Run
Service	→	Internal/External LB
Ingress	→	API Gateway, ALB
Job/Cron	→	Lambda, Batch

운영 기본기 · 확장성

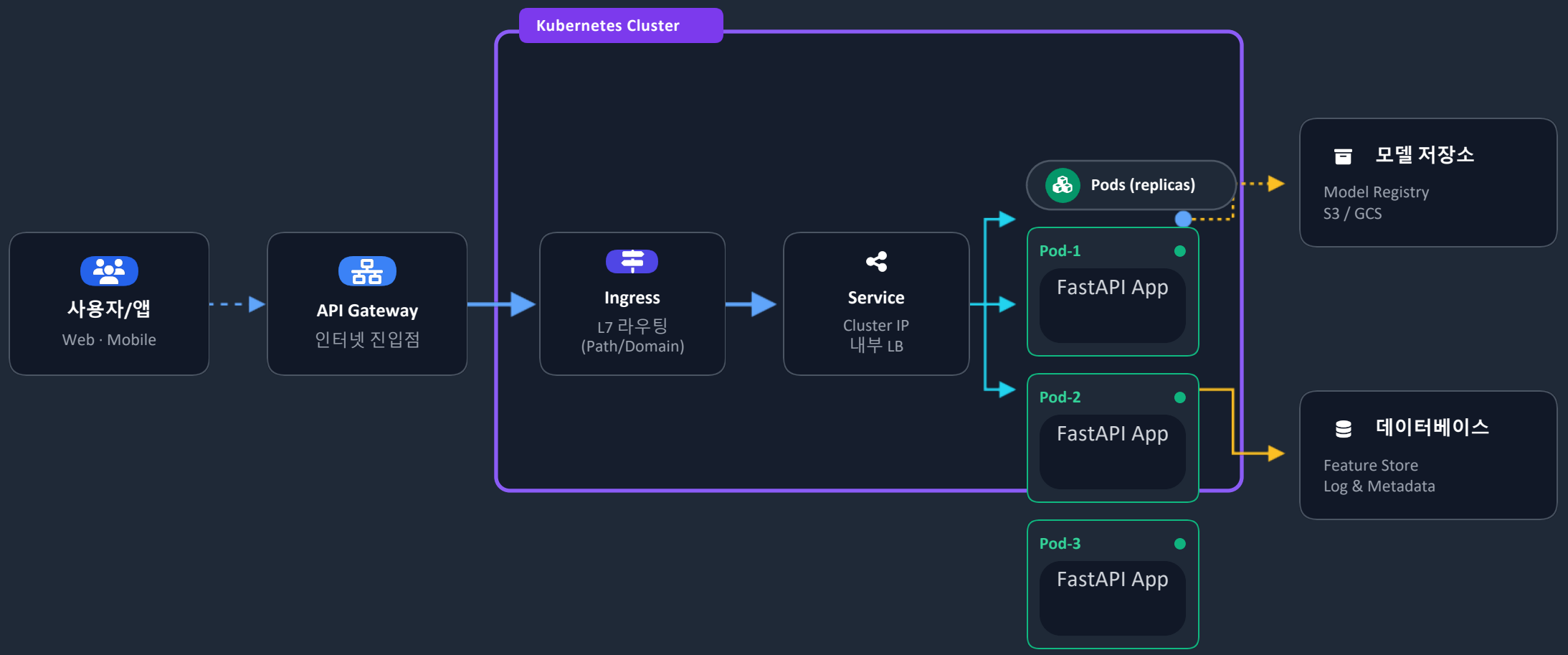
프로덕션 레벨의 고가용성(HA)과 무중단 배포를 위한 핵심 원리가 플랫폼 자체에 내장되어 있습니다.

Core Ops Features

- HPA/VPA (Auto Scale)
- Rolling Update
- Self-Healing
- Liveness/Readiness Probe
- Pod Disruption Budget
- ConfigMap/Secret

Kubernetes 기반 ML 서빙 아키텍처

사용자/앱 → API Gateway → Ingress → Service → Pods → 모델 저장소/DB



배포 시 실제 일어나는 일 (4단계)

1 코드 · 이미지 빌드

- Git 푸시로 CI 파이프라인 트리거
- Dockerfile로 컨테이너 이미지 생성

Example:
GitHub Actions가 docker build 실행

2 레지스트리 푸시

- 이미지 태깅: v2.1
- ECR/GCR 등 컨테이너 레지스트리에 저장

Path:
registry.example.com/ml-service:v2.1

3 Deployment 업데이트

- YAML에서 image 태그를 v2.1로 변경
- kubectl apply 또는 CI가 자동 적용

Method:
선언형(원하는 상태)으로 클러스터에 전달

4 롤링 업데이트

- Pod를 하나씩 교체하며 헬스체크
- 무중단으로 새 버전으로 전환

Result:
문제 발생 시 자동/수동 롤백 가능

- **CI 단계**: 소스 변경 → 이미지 빌드 → 레지스트리 푸시
- **CD 단계**: 매니페스트 적용 → 롤링 업데이트

핵심: 선언형(Deployment) 명세와 헬스체크 기반의 점진적 배포로, 서비스 중단 없이 안전하게 새 버전을 사용자에게 전달합니다.

최소 코드 아티팩트

Dockerfile · Kubernetes Deployment YAML

Dockerfile (앱 패키징)

컨테이너 이미지로 패키징하여 어디서나 동일한 환경 보장

```
FROM python:3.9-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
ENV PORT=8000
CMD ["uvicorn","main:app","--host","0.0.0.0","--port","8000"]
```

Deployment (무중단 배포)

원하는 상태 선언 → 레플리카 유지 · 롤링 업데이트

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ml-service
spec:
  replicas: 3
  selector:
    matchLabels:
      app: ml-service
  template:
    metadata:
      labels:
        app: ml-service
    spec:
      containers:
      - name: ml-service
        image: registry.example.com/ml-service:v2.1
        ports:
        - containerPort: 8000
        resources:
          requests: { cpu: "500m", memory: "512Mi" }
          limits:   { cpu: "1",    memory: "1Gi" }
```

모니터링 3축 (피라미드)

비즈니스 영향도부터 인프라 상태까지, 계층별 도구와 임계값을 설정하여 장애를 격리합니다.

3 비즈니스 모니터링 (Business KPI)

Google Analytics · Amplitude · Tableau

Alert 임계값 예시

CTR 급락> 10% Drop

전환율(CVR)< 2.0% Baseline

시나리오 진단

"매출이 줄었다면?" → 모델이 엉뚱한 상품을 추천했는지(Layer 2) 확인

2 모델 모니터링 (Model Performance)

MLflow · Evidently AI · Arize

Alert 임계값 예시

Accuracy Drop> 5%

Data Drift (PSI)> 0.2 (주의)

시나리오 진단

"정확도가 떨어졌다면?" → 계절성 변화나 입력 데이터 분포 변화(Drift) 체크

1 인프라 모니터링 (System Health)

Prometheus · Grafana · K8s Dashboard

Alert 임계값 예시

CPU / Memory> 80% Usage

Latency (p99)> 500ms

시나리오 진단

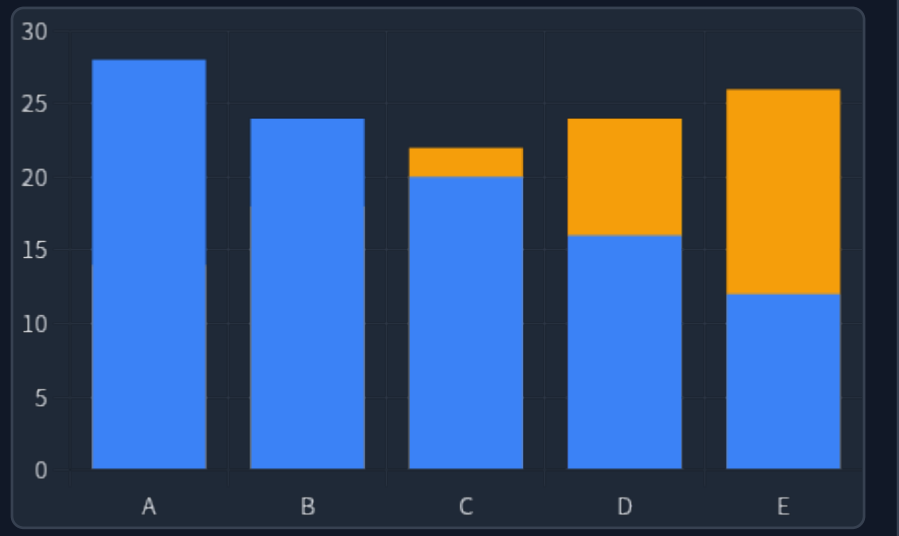
"응답이 느리다면?" → Pod 스케일링(HPA) 부족 또는 메모리 누수 확인

장애 대응 드릴다운(Drill-down): 사용자 불만 접수 → **비즈니스 지표 확인** → **모델 Drift 체크** → **CPU/Latency 점검**

데이터 드리프트 개념

무엇인지, 왜 발생하는지, 어떤 유형이 있는지 배경을 정리합니다.

입력 분포 변화(시각화)



예: 동일 피처의 분포가 계절/트렌드 변화로 이동
→ 예측 품질 저하 가능

정의

데이터 드리프트는 **운영 중 입력 데이터의 분포가 학습 시점과 달라지는 현상**으로,
모델 성능 저하를 유발할 수 있습니다.

왜 발생할까?

- 계절성/트렌드 변화
- 사용자/시장 변화
- 데이터 수집/전처리 변경
- 피처/스키마 변경

유형

- **데이터 드리프트** (입력 분포 변화)
- **컨셉 드리프트** (관계 변화)
- **레이블 분포 변화** (Target Shift)

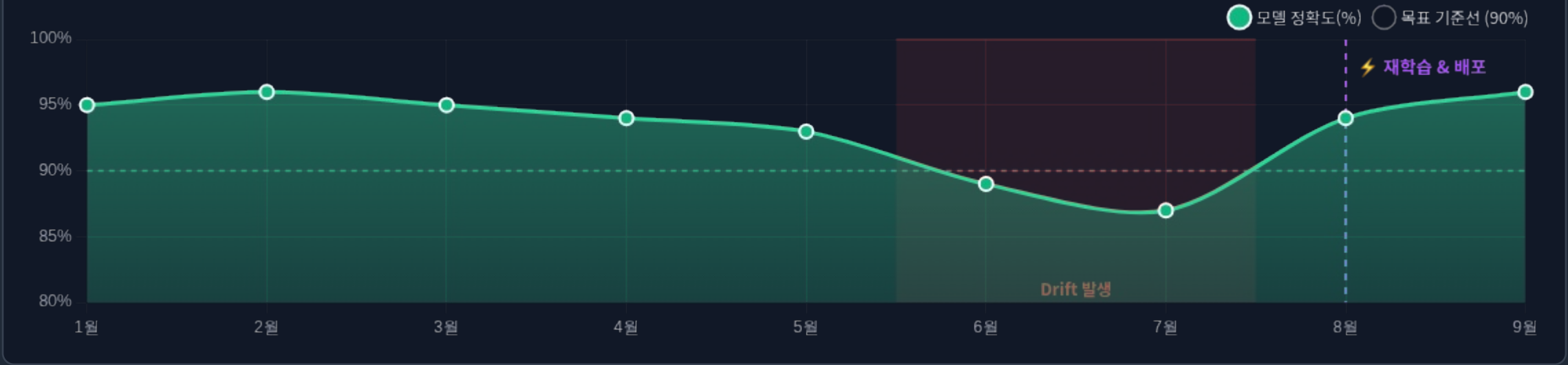
핵심: 드리프트는 **감지 → 원인 파악 → 재학습/피처 업데이트** 루프를 촉발합니다.

데이터 드리프트 사례: 겨울 → 여름

겨울 데이터로 학습한 모델이 여름에 성능 하락을 겪고, 재학습으로 회복한 흐름을 시각화합니다.

모델 정확도(Accuracy) 추이

정상 구간 · 드리프트(성능 저하) · 재학습/배포



1. 관측 (Monitor)

6월부터 정확도가 급격히 하락하여 **87%**까지 떨어짐. 임계치(90%) 이탈 알람 발생.

2. 분석 (Analyze)

겨울 의류 학습 모델에 여름 의류(반팔, 수영복) 데이터가 유입됨. **계절성 데이터 드리프트** 확인.

3. 대응 (Retrain)

최신 6~7월 데이터를 포함하여 모델 **재학습 및 배포**. 정확도 95% 수준으로 즉시 회복.

롤아웃 전략 3가지 심층 비교

Rolling Update · Blue-Green · Canary — 기술적 구현 사항과 운영 모니터링 포인트

Rolling Update
점진적 Pod 교체 (K8s Default)

v1 → v2 교체 중
Zero Downtime

🔧 구현 핵심 (Manifest)
- maxSurge: 추가 생성 허용 비율
- maxUnavailable: 0 설정 (무중단 필수)
- **ReadinessProbe**: 트래픽 유입 전 헬스체크 필수

📊 운영/모니터링
- 배포 중 HTTP 5xx 에러율 급증 확인
- v1/v2 공존 시 DB 스키마 호환성 주의

✅ **추천:** 일반적인 웹 서비스, 스테이트리스 앱

Blue-Green
트래픽 일괄 전환

Blue (Idle) ⇄ **Green (Live)**

🔧 구현 핵심 (Service/Ingress)
- **Service Selector** 교체 (Label 변경)
- 독립된 인프라 환경(더블 리소스) 준비
- 운영 환경과 동일한 Pre-prod 테스트 가능

📊 운영/모니터링
- **즉시 롤백** 가능 (스위치만 원복)
- Long-running 세션 및 데이터 동기화 이슈

✅ **추천:** 결제 시스템 등 핵심/대규모 서비스

Canary
트래픽 분할 & 검증

v1 (Stable) **v2 (Canary)**

10% Traffic → Analysis

🔧 구현 핵심 (Mesh/Controller)
- **Istio/Linkerd**: 가중치 기반 라우팅
- **Argo Rollouts**: Step별 트래픽 자동 증량
- 헤더 기반(Internal User) 타겟팅 가능

📊 운영/모니터링
- **Golden Signals** 비교 (Latency, Error)
- 지표 악화 시 자동 중단/롤백 설정 필수

✅ **추천:** ML 모델 배포, 불확실성이 높은 변경

자동화와 가드레일 (Safety Guardrails)

운영 환경 보호를 위한 4단계 안전 장치와 기술적 구현 상세

1 자동 트리거

성능 저하 및 데이터 변화 감지 시 사람 개입 없이 파이프라인 실행

기술 구현 (Implementation)

- **Monitoring:** Prometheus + Grafana Alerting
- **Orchestration:** Airflow DAG / Kubeflow Pipeline 트리거 (Webhook)

임계값 예시

Drift PSI Score > 0.2

Accuracy Drop > 5%

2 검증 (Evaluation)

신규 모델이 배포 가능한 품질인지 자동화된 테스트로 확인

기술 구현

- **Dataset:** 학습에 쓰지 않은 Hold-out 셋 또는 최신 1주 데이터
- **Code:** PyTest + MLflow 모델 평가 API 연동

통과 조건 (Gate)

New AUC ≥ Current Baseline

Inference Latency (p99) < 200ms

3 승인 게이트

자동 검증 통과 후, 정책 또는 책임자에 의한 최종 배포 승인

기술 구현

- **CI/CD:** GitHub Actions Environments / GitLab Manual Job
- **Ops:** Slack Bot 알림 → '승인' 버튼 클릭 시 배포 진행

승인 워크플로

Model Card(성능 리포트) 자동 첨부

Shadow 배포 결과 확인 후 승인

4 점진적 롤아웃

트래픽을 조금씩 늘리며 실전 검증, 이상 발생 시 즉시 차단

기술 구현

- **Deployment:** Argo Rollouts / Flagger (Istio 연동)
- **Strategy:** Canary Analysis (1m 간격 트래픽 증량)

자동 롤백 조건

HTTP 5xx Rate > 1%

Latency(p99) > 500ms

Automated Safety First

속도를 위한 자동화(Automation)와 안전을 위한 가드레일(Guardrails)의 균형이 지속 가능한 MLOps의 핵심입니다.

Airflow MLflow Argo

ML 팀 구조: 여러분은 어디에 있나요?

DS · ML/백엔드 · MLOps/Platform · 프론트엔드/앱 · SRE — 역할과 연결 지점 한눈에 보기

Data Scientists
모델 연구·실험, 피처 엔지니어링, 오프라인 평가
Jupyter | Python/TF/PT

ML Engineers / Backend
서빙 API 구현, 피처/데이터 파이프라인, 모델 레지스트리 연동
FastAPI | SQL

MLOps / Platform
컨테이너·Kubernetes 운영, CI/CD, 관측성(로그·메트릭·트레이스)
Docker/K8s | CI/CD

Frontend / App
UX 구현, /predict 등 API 연동, 결과 시각화 · 에러 핸들링
React / React Native | Fetch/Axios

SRE / DevOps
신뢰성·가용성 보장, 알림·온콜, 용량 계획 및 스케일링
SLI/SLO | Alerts

핵심: 프론트엔드는 사용자와 모델 사이의 마지막 연결 지점입니다. 문제 발생 시 어디(프론트·API·인프라·모델)를 먼저 확인할지 이해하는 것이 협업의 시작입니다.

커리어 경로 3가지 (장기 성장 관점)

ML 애플리케이션 엔지니어

프론트 강점을 살려 서비스로 확장

장기 여정 | 균형형

- 가치: 프로토타입을 “서비스”로 연결하는 실행력
- 성장 포인트: API 설계, 데이터 계약, 관측성 기초
- 적합: 프론트 경험을 살려 백엔드/ML을 잇고 싶은 분

프론트엔드 + AI 전문가

사용자 가치 중심의 AI 제품화

제품 임팩트

- 가치: AI를 자연스럽게 UX에 녹여 사용자 성과 향상
- 성장 포인트: 응답 UX, 성능/캐싱, A/B 실험 문화
- 적합: UX·프로덕트 감각이 강한 프론트엔드

MLOps 엔지니어

플랫폼·시스템으로 신뢰성 구축

장기 여정 | 심화/플랫폼

- 가치: 스케일·가용성·자동화를 통해 팀 생산성 향상
- 성장 포인트: Kubernetes, CI/CD, IaC, 관측성
- 적합: 시스템 설계와 운영을 즐기는 엔지니어

첫걸음 가이드와 핵심 메시지

첫걸음 가이드 예시

- 수업에서 만든 간단한 모델을 선택해 기록(버전 이름)을 붙여보기
- FastAPI 예제 따라 하며 /predict 엔드포인트 하나 만들어보기
- Dockerfile 기본 패턴 익히고 로컬에서 컨테이너 실행해보기
- Cloud Run 또는 Minikube 중 편한 방법으로 간단 배포 체험
- 로그에 입력·응답을 남기고, 지연시간을 콘솔에서 확인해보기
- React에서 /predict 호출해 간단한 UI로 결과를 시각화

핵심 메시지 3가지

1. MLOps는 새로운 유행이 아니라,
   익숙한 소프트웨어 관행을 ML에 확장한 것입니다.
2. 서비스는 루프입니다: 모니터링 → 개선 → 재배포.
   한 번에 완벽보다 점진적 향상이 중요합니다.
3. 나의 강점(프론트, UX 등) 위에 한 걸음을 더하면,
   커리어는 꾸준히 확장됩니다.

Thank You

감사합니다

오늘 귀한 시간을 내어 함께해 주셔서 진심으로 감사드립니다.
여러분의 커리어를 응원합니다!

Q & A
궁금한 점이 있으신가요? 지금 질문해 주세요.